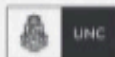


CERTIFICACIÓN UNIVERSITARIA



UNC



FCEFYN



Manual para el alumno

Gitlab CI / Github
Actions II



**WE WANT
SKILLS!**

mundosE

Manual para el Alumno: Gitlab CI / Github Actions I.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
1. Introducción a GitHub Actions.....	3
2. Estructura de un archivo YAML de workflow.....	4
3. Eventos que disparan workflows.....	5
4. Organización del workflow: Jobs, Steps y Actions.....	7
5. Manejo de variables y secretos.....	8
6. Runners hospedados y self-hosted.....	10
7. Comparación GitLab CI vs GitHub Actions.....	11
Bibliografía.....	13

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En él encontraremos un resumen de cada tema tratado en el encuentro, así como también recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el Alumno: Gitlab CI / Github Actions II

Objetivo del encuentro:

El objetivo de este encuentro es que los estudiantes comprendan el funcionamiento general de GitHub Actions como plataforma de automatización CI/CD integrada a GitHub. Se busca que aprendan a escribir y configurar workflows en formato YAML, identifiquen los eventos que disparan estos procesos, comprendan cómo se organizan las ejecuciones en jobs, steps y actions, y sepan gestionar variables y secretos de forma segura. Además, deben diferenciar entre runners hospedados y self-hosted, entendiendo cuándo conviene utilizar cada uno, y finalmente evaluar de forma comparativa las características de GitHub Actions frente a GitLab CI/CD.

Contenido a desarrollar:

- Introducción a GitHub Actions
- Estructura de un archivo YAML de workflow
- Eventos que disparan workflows
- Organización del workflow: Jobs, Steps y Actions
- Manejo de variables y secretos
- Runners hospedados y self-hosted
- Comparación GitLab CI vs GitHub Actions

1. Introducción a GitHub Actions

GitHub Actions es una herramienta de integración continua (CI) y entrega continua (CD) desarrollada por GitHub, que permite automatizar tareas dentro del ciclo de vida del desarrollo del software. Esta plataforma permite crear flujos de trabajo (workflows) que se ejecutan automáticamente cuando ocurren ciertos eventos dentro del repositorio, como la creación de un *pull request*, la actualización de una rama, o incluso de forma programada. Estos workflows están definidos en archivos YAML que se ubican dentro de un directorio específico del repositorio ([.github/workflows/](#)).

Una de las principales ventajas de GitHub Actions es que está completamente integrada con la interfaz de GitHub, lo que facilita su adopción sin necesidad de herramientas adicionales. Su diseño permite aprovechar el ecosistema ya existente de GitHub, incluyendo su control de versiones, colaboración con otros desarrolladores, y acceso a una comunidad activa que mantiene cientos de *actions* disponibles en el marketplace.

GitHub Actions se basa en una arquitectura flexible que permite definir múltiples *jobs* (trabajos), organizados en pasos (*steps*), donde cada paso puede ejecutar una tarea o utilizar una acción reutilizable. Gracias a esta estructura modular, es posible construir flujos simples como la ejecución de pruebas unitarias, o complejos como pipelines de despliegue a múltiples entornos.

Además, GitHub Actions permite usar *runners* hospedados (infraestructura proporcionada por GitHub) o *self-hosted* (máquinas propias) para ejecutar los workflows. Esta dualidad proporciona tanto facilidad de uso como flexibilidad avanzada para entornos más personalizados.

La plataforma es gratuita para repositorios públicos, lo que la hace ideal para proyectos de código abierto, y ofrece planes con minutos incluidos para proyectos privados. Esta política ha incentivado su adopción tanto por parte de individuos como de organizaciones.

Algunos casos de uso típicos de GitHub Actions incluyen:

- Automatizar la ejecución de pruebas con cada *push*.
- Generar y desplegar documentación automáticamente.
- Publicar paquetes a npm, PyPI u otros gestores.
- Realizar análisis estático de código y revisión de estilo.

En resumen, GitHub Actions proporciona un entorno poderoso, escalable y bien documentado que permite automatizar tareas repetitivas del desarrollo con una curva de aprendizaje baja, especialmente para quienes ya utilizan GitHub como plataforma de control de versiones.

Recursos para profundizar:

- Video: [GitHub Actions Explained](#)
- Documentación oficial: <https://docs.github.com/en/actions/learn-github-actions>
- Blog: [Introduction to GitHub Actions](#)

2. Estructura de un archivo YAML de workflow

Un archivo de workflow en GitHub Actions está escrito en formato YAML y debe guardarse en el directorio `.github/workflows/` del repositorio. Cada archivo describe un flujo de trabajo que se activa en función de eventos específicos y está compuesto por una serie de *jobs* y *steps* que definen las tareas a ejecutar.

La estructura básica de un archivo YAML de GitHub Actions incluye:

```
name: CI Workflow

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Instalar dependencias
        run: npm install
      - name: Ejecutar pruebas
        run: npm test
```

Explicación de las secciones:

- **name:** Es un nombre opcional que identifica el workflow.
- **on:** Define el o los eventos que disparan la ejecución del workflow. Puede ser `push`, `pull_request`, `workflow_dispatch`, `schedule`, etc.
- **jobs:** Cada *job* representa una unidad de ejecución independiente. Dentro de cada *job*:
 - **runs-on:** Especifica el sistema operativo del *runner* (como

`ubuntu-latest, windows-latest`, etc.).

- **steps**: Define los pasos a ejecutar en orden secuencial.

Cada *step* puede utilizar:

- **uses**: Para ejecutar una acción predefinida del marketplace.
- **run**: Para ejecutar comandos directamente en el entorno del *runner*.

La modularidad del YAML permite crear workflows reutilizables, integrando herramientas externas o incluso otros repositorios. También se pueden definir variables de entorno, utilizar condicionales (`if:`), agrupar trabajos en paralelo o secuencia, y usar estrategias de matrices para probar múltiples configuraciones.

Los errores más comunes al escribir archivos YAML están relacionados con la indentación, ya que YAML es muy sensible a los espacios. Por eso, se recomienda usar un editor con soporte de sintaxis YAML y validar los workflows mediante pruebas incrementales.

Recursos para profundizar:

- Guía YAML oficial de GitHub:
<https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions>
- Artículo: [Writing Your First GitHub Action YAML](#)

3. Eventos que disparan workflows

GitHub Actions permite automatizar flujos de trabajo a partir de eventos específicos que ocurren dentro del repositorio. Estos eventos son señales que activan la ejecución de un workflow y representan una de las mayores fortalezas de esta herramienta, ya que permiten integrar directamente las automatizaciones con la lógica del desarrollo y colaboración dentro del proyecto.

El bloque `on:` en el archivo YAML define los eventos que activan el workflow. Algunos de los más comunes son:

- **push**: se activa cuando se realiza un *push* a una rama.
- **pull_request**: se activa cuando se abre, actualiza o cierra un *pull request*.
- **workflow_dispatch**: permite ejecutar el workflow manualmente desde la interfaz de GitHub.
- **schedule**: permite programar la ejecución automática usando sintaxis cron.
- **release, issues, watch**, entre muchos otros.

Ejemplo básico:

```
on:
  push:
    branches:
      - main
```

Este ejemplo indica que el workflow se ejecutará cada vez que se haga un push a la rama **main**.

También se pueden combinar eventos o usar filtros más avanzados para reducir ejecuciones innecesarias:

```
on:
  push:
    branches:
      - main
    paths:
      - 'src/**'
```

Este caso solo dispara el workflow si los archivos dentro de **src/** son modificados.

workflow_dispatch agrega una interfaz de botón dentro de la pestaña *Actions* de GitHub, muy útil para tareas manuales como despliegues controlados. Incluso es posible definir parámetros de entrada para que el usuario seleccione opciones antes de ejecutar:

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Seleccionar entorno'
        required: true
        default: 'staging'
```

El evento **schedule** se define con una expresión cron:

```
on:
  schedule:
    - cron: '0 8 * * *' # todos los días a las 08:00 UTC
```

Esto permite ejecutar tareas como limpieza de caché o backups de forma programada.

Recursos para profundizar:

- Documentación de eventos soportados:
<https://docs.github.com/en/actions/using-workflows/events-that-trigger-workflows>
- Ejemplos avanzados de eventos:
<https://docs.github.com/en/actions/using-workflows/triggering-a-workflow>

4. Organización del workflow: Jobs, Steps y Actions

Los flujos de trabajo de GitHub Actions se estructuran en tres niveles principales: **jobs**, **steps** y **actions**. Comprender la jerarquía entre ellos es fundamental para construir pipelines limpios, modulares y eficientes.

- Un **job** es una unidad de ejecución completa, que puede correr en paralelo o en secuencia respecto a otros jobs. Se ejecuta en su propio runner y puede estar compuesto por múltiples pasos.
- Un **step** es una acción o conjunto de comandos individuales dentro de un job. Los steps se ejecutan secuencialmente y comparten el mismo entorno dentro del job.
- Una **action** es un módulo reutilizable que encapsula tareas específicas. Puede estar publicada en el GitHub Marketplace o desarrollarse localmente.

Ejemplo básico:

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Instalar dependencias
        run: npm install
      - name: Ejecutar pruebas
        run: npm test
```

En este ejemplo:

- Hay un único job llamado **build**.
- El job se ejecuta en un runner Ubuntu.
- Dentro del job hay tres pasos: uno para hacer checkout del código, otro para instalar dependencias, y un tercero para correr las pruebas.

Los jobs pueden ejecutarse en paralelo de forma predeterminada. Si uno depende del resultado de otro, se puede usar **needs**:

```
jobs:
  build:
  |   runs-on: ubuntu-latest
    steps: [...]

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps: [...]
```

Esto garantiza que **deploy** solo se ejecute si **build** finaliza correctamente.

Las actions pueden ser de tres tipos:

1. **Acciones del marketplace** (como **actions/setup-node**)
2. **Acciones locales** creadas dentro del repositorio.
3. **Acciones Dockerizadas**, que permiten encapsular entornos.

Crear y reutilizar actions es clave para mantener consistencia, compartir lógica entre proyectos y reducir errores por código repetido.

Recursos para profundizar:

- Documentación oficial de Actions:
<https://docs.github.com/en/actions/creating-actions/about-custom-actions>
- Ejemplos en GitHub Marketplace:
<https://github.com/marketplace?type=actions>

5. Manejo de variables y secretos

GitHub Actions permite utilizar variables y secretos para manejar datos que deben ser reutilizables o confidenciales dentro de los workflows. El uso correcto de estas entidades no solo aporta claridad y orden a los scripts, sino que también asegura la protección de información sensible.

Las **variables** en GitHub Actions pueden declararse directamente en el archivo YAML o a través de configuraciones del repositorio. Las variables locales se declaran bajo **env** y pueden aplicarse a todo un workflow, a un job o a un step específico. Por ejemplo:

```
env:  
  NODE_ENV: 'production'
```

Este bloque define `NODE_ENV` como variable global en el entorno de ejecución. Estas variables se pueden invocar luego con `${{ env.NOMBRE_VARIABLE }}`.

Existen también variables predefinidas de contexto como `github.repository`, `github.actor`, o `github.ref` que proporcionan información sobre el entorno y el evento que disparó el workflow.

Los **secretos** son aún más importantes, ya que contienen información sensible como tokens de autenticación, claves API, contraseñas u otros valores que no deben ser expuestos. Estos se gestionan desde la sección **Settings > Secrets and variables** en GitHub. Para utilizarlos dentro de un workflow, se accede mediante:

```
env:  
  TOKEN: ${{ secrets.MI_TOKEN }}
```

Durante la ejecución del workflow, los secretos son ocultados automáticamente en los logs. Si se intenta imprimir un secreto, GitHub reemplazará su contenido con `***` para evitar filtraciones accidentales.

Es posible definir secretos a nivel de repositorio, organización o entorno. Esta jerarquía permite controlar mejor el acceso y reutilizar configuraciones entre diferentes proyectos.

Buenas prácticas al usar variables y secretos:

- Usar nombres descriptivos y consistentes.
- No codificar directamente valores sensibles en los YAML.
- Revocar y rotar periódicamente los secretos.
- Utilizar distintos secretos para distintos entornos (dev, staging, prod).

Recursos para profundizar:

- Documentación:
<https://docs.github.com/en/actions/security-guides/encrypted-secrets>
- Hub Secrets: The Basics and 4 Critical Best Practices:
<https://configu.com/blog/github-secrets-the-basics-and-4-critical-best-practices/>

6. Runners hospedados y self-hosted

Los *runners* son las máquinas que ejecutan los workflows definidos en GitHub Actions. Existen dos tipos: **runners hospedados por GitHub** y **runners self-hosted (autoalojados)**.

Los **runners hospedados** son máquinas virtuales mantenidas por GitHub. Están disponibles en distintos sistemas operativos (**ubuntu-latest**, **windows-latest**, **macos-latest**) y se aprovisionan automáticamente cada vez que se dispara un workflow. Estos runners se eliminan al finalizar el job, asegurando un entorno limpio y seguro para cada ejecución.

Ventajas de los runners hospedados:

- Configuración automática.
- Entorno limpio y preinstalado con herramientas comunes.
- Ideal para comenzar con flujos simples o para equipos pequeños.

Limitaciones:

- Restricción de minutos gratuitos en repositorios privados.
- No permite instalar software personalizado ni acceder a recursos locales.

Los **runners self-hosted** son máquinas físicas o virtuales administradas por el usuario. Se registran manualmente al repositorio, y una vez configurados, ejecutan jobs como lo haría cualquier otro runner.

Son útiles en escenarios donde:

- Se requiere software específico o dependencias no disponibles en los runners hospedados.
- Es necesario ejecutar tareas que interactúan con redes privadas o bases de datos internas.
- Se busca optimizar el uso de recursos para evitar los límites de minutos en GitHub.

Para registrar un runner self-hosted:

1. Se crea un token desde la interfaz de GitHub.
2. Se descarga el agente correspondiente al sistema operativo.
3. Se ejecuta el script de configuración con los parámetros adecuados.

Los runners self-hosted ofrecen flexibilidad y control, pero también exigen mantener medidas de seguridad, actualizaciones periódicas y monitoreo continuo.

Recursos para profundizar:

- Configuración de runners:
<https://docs.github.com/en/actions/using-github-hosted-runners/about-github-hosted-runners>
- Guía para runners self-hosted:
<https://docs.github.com/en/actions/hosting-your-own-runners/about-self-hosted-runners>

7. Comparación GitLab CI vs GitHub Actions

GitHub Actions y GitLab CI/CD son dos de las herramientas de automatización más utilizadas en el desarrollo moderno. Ambas permiten implementar pipelines CI/CD mediante archivos YAML y ofrecen entornos robustos para ejecutar pruebas, compilar software y hacer despliegues automáticos.

GitHub Actions destaca por:

- Estar profundamente integrado en la interfaz de GitHub.
- Contar con un marketplace muy activo con miles de acciones reutilizables.
- Su simplicidad para configurar workflows básicos.
- Facilidad de uso para quienes ya trabajan en repositorios GitHub.

GitLab CI/CD tiene como principales ventajas:

- Integración nativa con todo el ciclo DevOps: control de versiones, issues, seguridad, despliegues.
- Mayor personalización en la definición de etapas y runners.
- Capacidades avanzadas como escaneo SAST/DAST sin plugins externos.

Comparación general:

Característica	GitHub Actions	GitLab CI/CD
Integración	GitHub	GitLab
Runners hospedados	Sí	Sí

Runners self-hosted	Sí	Sí
Marketplace	Extenso	Limitado
Seguridad integrada	Básica (mediante secretos)	Avanzada (SAST/DAST)
Experiencia de usuario	Integrada a Pull Requests	Pipeline gráfico detallado

En la práctica, GitHub Actions es más recomendable si se trabaja exclusivamente en el ecosistema GitHub y se prioriza la facilidad de uso. GitLab CI/CD resulta más potente para proyectos empresariales que requieren una solución todo-en-uno.

Recursos para profundizar:

- Artículo actualizado:
<https://www.bytebase.com/blog/gitlab-ci-vs-github-actions/>

Bibliografía

García, M. (2021). *Introducción a Cloud Computing: Modelos de Servicios, Implementaciones y Estrategias Empresariales*. Ed. Alfaomega. ISBN: 978-6075385687.

Google Cloud Platform Documentation. (2024). *Documentación Oficial de Google Cloud Platform*. Google LLC.
Disponible en: <https://cloud.google.com/docs?hl=es-419>

Google Cloud Compute Engine Documentation. (2024). *Compute Engine – Máquinas virtuales en GCP*. Google LLC.
Disponible en: <https://cloud.google.com/compute/docs?hl=es-419>

Google Cloud SQL Documentation. (2024). *Bases de datos gestionadas en Google Cloud*. Google LLC.
Disponible en: <https://cloud.google.com/sql/docs?hl=es-419>

Google Cloud Run Documentation. (2024). *Ejecución serverless de contenedores*. Google LLC.
Disponible en: <https://cloud.google.com/run/docs?hl=es-419>

Google Cloud Storage Documentation. (2024). *Almacenamiento de objetos en la nube*. Google LLC.
Disponible en: <https://cloud.google.com/storage/docs?hl=es-419>

Google BigQuery Documentation. (2024). *Servicio de análisis de datos a gran escala*. Google LLC.
Disponible en: <https://cloud.google.com/bigquery/docs?hl=es-419>

Rosenberg, J. & Mateos, J. (2022). *Practical Google Cloud Platform: Harnessing Google's Infrastructure for Business Success*. Ed. Apress. ISBN: 978-1484274960.

Joshi, S. (2020). *Cloud Computing: Concepts, Technology & Architecture*. Ed. Pearson Education. ISBN: 978-9353947051.

Stackscale (2023). *Modelos de servicio Cloud Computing*. Stackscale Blog.
Disponible en: <https://www.stackscale.com/es/blog/modelos-de-servicio-cloud>